

BUG1

```
Manifest [Java Application] C:\Users\Administrator\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.w
Flight open for ticket sales.
Seating plan has 14 seats.
Do you want to purchase a ticket? Yes
Do you want BUSINESS class? Y
First name? brill
Last name? torino
Initial? bt
Are you a frequent flyer? N
Are you a frequent flyer?. Please answer Y or N: N
Are you a frequent flyer?. Please answer Y or N: . Please answer Y or N: N
Are you an airline employee? N
Are you an airline employee?. Please answer Y or N: N
Are you an airline employee?. Please answer Y or N: . Please answer Y or N: N
Ticket issued: 1000001 for seat 1B at $750.0
Do you want to purchase a ticket?
```

- Where (class/method names): `com.cc.airline.utilities.UserPrompter.getYesNoAnswer()`
- User input: **At any yes/no prompt (e.g., “Are you a frequent flyer?”), the user enters N (or n) once.**
- System response: **The program re-prompts 1–3 times with “Please answer Y or N” instead of accepting the input and proceeding.**
- Expected (what should have happened): **The program should accept N/n immediately and return false without repeating the prompt.**
- Actual root cause (from code): **Inside `getYesNoAnswer()`, only 'Y' is treated as valid; 'N' falls through to the re-prompt logic.**

FIXED

```
Manifest [Java Application] C:\Users\Administrator\p2\pool\plugins\or
Flight open for ticket sales.
Seating plan has 14 seats.
Do you want to purchase a ticket? y
Do you want BUSINESS class? n
Do you want ECONOMY class? Y
First name? brill
Last name? torino
Initial? bt
Are you a frequent flyer? N
Are you an airline employee? N
Ticket issued: 1000001 for seat 3D at $500.0
Do you want to purchase a ticket?
```

- The bug was found in the `UserPrompter` class, inside the method: `public boolean getYesNoAnswer()`

```
public boolean getYesNoAnswer() {
    for (int i = 0; i < 3; i++) {
        String answer = getAnswer();
        if (answer == null)
            return false;
        char ans = answer.toUpperCase().charAt(0);
        if (ans == 'Y')
            return true;

        //start added
        if (ans == 'N')
            return false;
        //end

        setPrompt(getPrompt() + ". Please answer Y or N: ");
    }
    return false;
}
```

BUG2

- Where (class/method names): `com.cc.airline.utilities.SeatReserver.sellTicket(SeatingClass)`
- User input:

```
Console X Problems Debug Shell
<terminated> Manifest [Java Application] C:\Users\Administrator\p2\pool
Flight open for ticket sales.
Seating plan has 14 seats.
Do you want to purchase a ticket? y
Do you want BUSINESS class? y
First name? garam
Last name? yoon
Initial? gy
Are you a frequent flyer? n
Are you an airline employee? n
Ticket issued: 1000001 for seat 1B at $750.0
Do you want to purchase a ticket? n
```

- System response: Console prints: *"Ticket issued: 1000001 for seat 1B at \$750.0"* (ticket looks correct). But in the final manifest, seat 1B still appears as *"Available."*

```
Flight manifest
1A Available
1B Available
2A Available
2B Available
2C Available
2D Available
3A Available
3B Available
3C Available
3D Available
4A Available
4B Available
4C Available
4D Available
```

- Expected result: The manifest should show seat 1B with the passenger's name, not *"Available."*
- Actual root cause (from code): In `sellTicket(...)` a new Ticket is created but never attached to the Seat:

```
Ticket ticket = new Ticket(passenger, seat, price);
// important step omitted here
System.out.println("Ticket issued: " + ticket);
System.out.println();
return true;
```

FIXED:

- After creating the ticket, attach it to the seat object:

```
95
96 Ticket ticket = new Ticket(passenger, seat, price);
97 // important step omitted here
98 seat.setTicket(ticket); //added this line
99 System.out.println("Ticket issued: " + ticket);
100 System.out.println();

Console X Problems Debug Shell
<terminated> Manifest [Java Application] C:\Users\Administrator\p2\pool\plugins\org.eclipse.justj.openjdk
Do you want to purchase a ticket? y
Do you want BUSINESS class? y
First name? garam
Last name? yoon
Initial? gy
Are you a frequent flyer? n
Are you an airline employee? n
Ticket issued: 1000001 for seat 1B at $750.0

Do you want to purchase a ticket? n
Flight manifest
1A Available
1B garam gy yoon
2A Available
2B Available
2C Available
2D Available
3A Available
3B Available
3C Available
3D Available
4A Available
4B Available
4C Available
4D Available
```

BUG3

- Where (class/method names):
com.cc.airline.ticketing.Discountable.discountPrice(double price)
com.cc.airline.utilities.SeatReserver.sellTicket (SeatingClass sClass)
com.cc.airline.passengers.StaffPassenger.discountPrice(double price)
- User input:

```
Console X
UserPrompter [Java Application] C:\Users\rkfa\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_2
Flight open for ticket sales.
Seating plan has 14 seats.
Do you want to purchase a ticket? Y
Do you want BUSINESS class? Y
First name? Garam
Last name? Yoon
Initial? GRY
Are you a frequent flyer? N
Are you an airline employee? Y
Employee ID? 2
```

- System response:

Ticket issued: 1000001 for seat 1A at \$375.0

Name	Value
discountPrice() return	375.0
> this	SeatReserver (id=27)
> sClass	SeatingClass (id=38)
> price	375.0

- Expected result: **\$750**
- Actual root cause: **The original discountPrice method only accepted price, so the code had no way to know the passenger's seating class. As a result, the discount was always applied, regardless of class.**

FIXED

BUG: The bug is that the discountPrice method only receives the ticket price and has no information about the seating class, so the code applies a 50% discount in all cases instead of limiting it to ECONOMY class only.

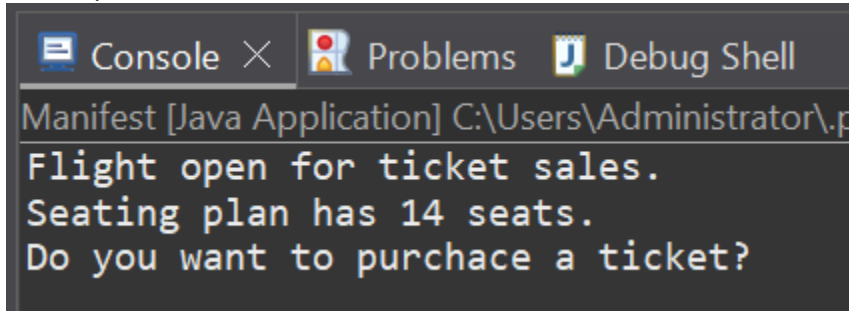
```
UserPrompte... Manifest.java UserPrompte... SeatingClas... StaffPasseng... *SeatReserve... X 40
88 Seat seat = assignSeat(sClass);
89 if (seat == null) {
90     System.out.println("Unable to assign seat, flight closed.");
91     return false;
92 }
93 if (passenger instanceof Discountable) {
94     price = ((Discountable) passenger).discountPrice(price, sClass); //fix 2 - deliver Class argument
95 }
96 Ticket ticket = new Ticket(passenger, seat, price);
97 // important step omitted here
98 System.out.println("Ticket issued: " + ticket);
99 System.out.println();
100 return true;
101 }
102 }

1 package com.cc.airline.ticketing;
2
3 public interface Discountable {
4     double discountPrice(double price, SeatingClass sClass); //fix 2
5 }
6

UserPrompte... Manifest.java UserPrompte... SeatingClas... *StaffPasse... X 40
8     private String employeeId;
9
10 public StaffPassenger( PassengerName pName, String employeeId) {
11     super(pName);
12     this.employeeId = employeeId;
13 }
14
15 public double discountPrice(double price) {
16     if (sClass == SeatingClass.ECONOMY) {
17         return price / 2.0; // fix 2 - only Economy is discountable
18     }
19     return price;
20 }
21
22 public String getEmployeeId() {
23     return employeeId;
```

```
UserPrompter [Java Application] C:\Users\rkfka\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.7.v
Last name? Yoon
Initial? GRY
Are you a frequent flyer? N
Are you an airline employee? Y
Employee ID? 2
Ticket issued: 1000001 for seat 1A at $750.0
```

- Where (class/method names): **com.cc.airline.utilities.Manifest.main(String[] args)**
- User input:



The screenshot shows the Eclipse IDE's Console window. The title bar includes icons for Console, Problems, and Debug Shell. The console text is as follows:

```
Manifest [Java Application] C:\Users\Administrator\p
Flight open for ticket sales.
Seating plan has 14 seats.
Do you want to purchace a ticket?
```

- System response: The word *"purchace"* is misspelled.
- Expected result: The prompt should read: Do you want to purchase a ticket?

Briefly define the term **Step in, Step out, Step over** and **Breakpoint**: (2)

Step In: a debugging command that executes the current line and, if a method is called, enters that method to debug it line by line.

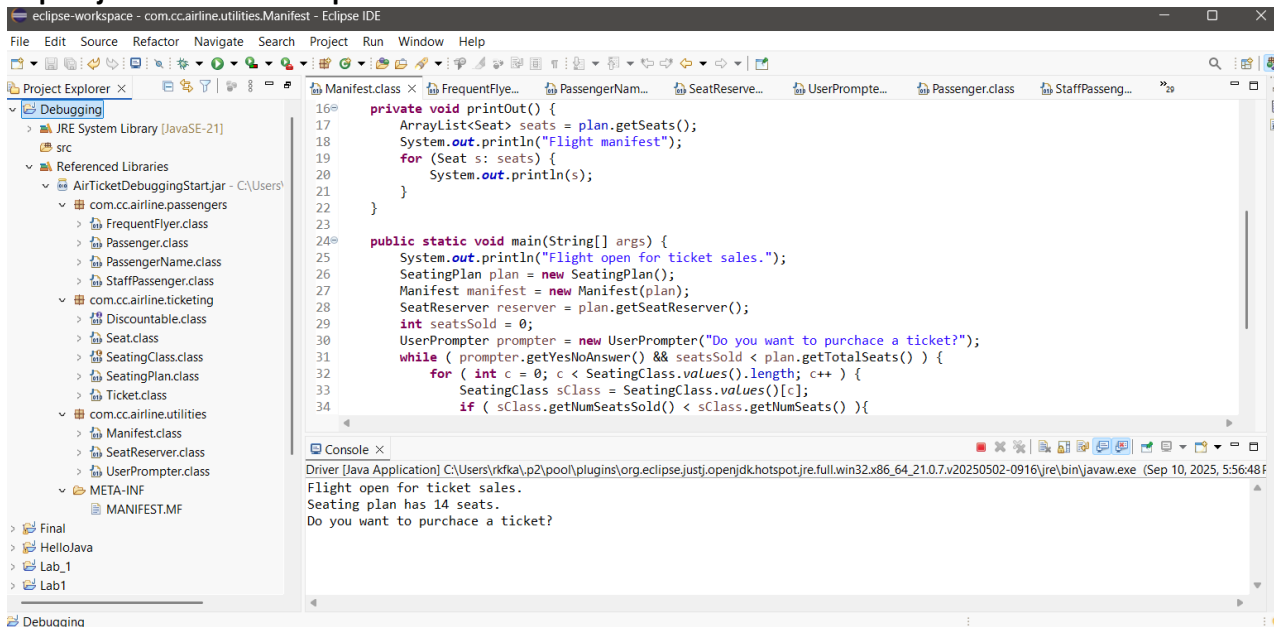
Step Over: a debugging command that executes the current line but skips entering called methods, moving to the next line in the current method.

Step Out: a debugging command that completes the rest of the current method and returns to the calling method, stopping after the method call.

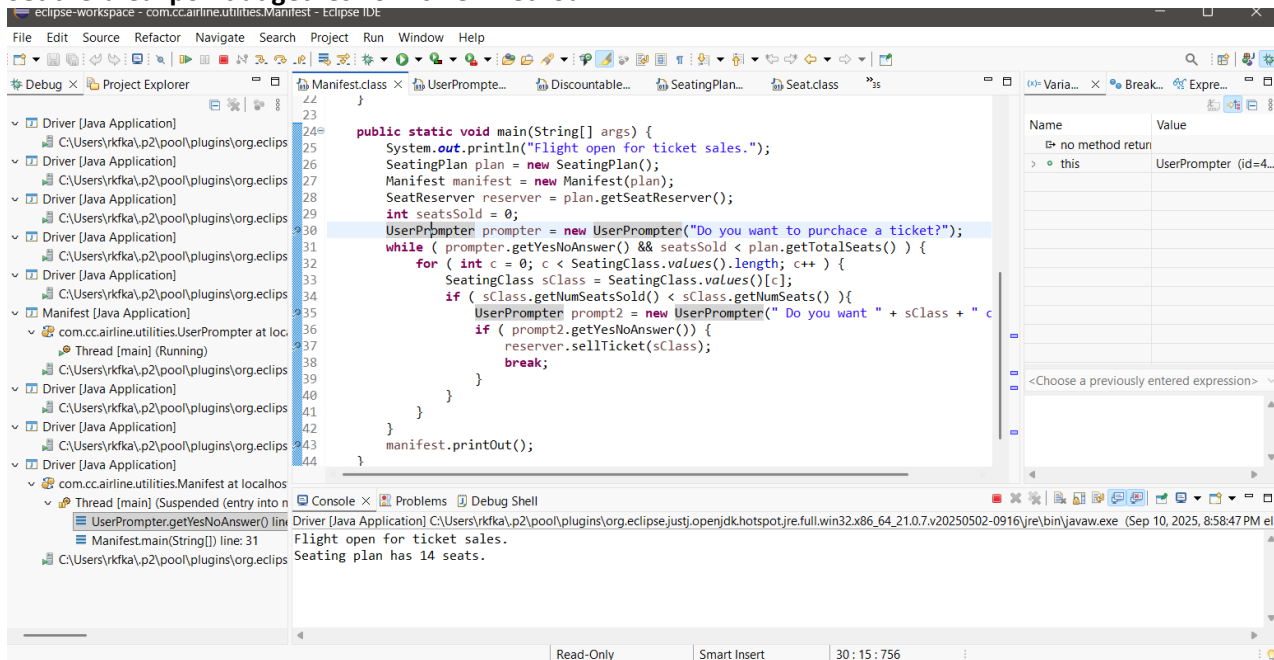
Breakpoint: a marker in the code that pauses execution at a specific line, allowing the programmer to inspect variables and program flow.

Give the Screenshots for the import of jar file into the Eclipse IDE , run the application in Debug mode and fix of the bug demonstrated in the class. (1)

Import jar file into the Eclipse IDE



Set the breakpoint at getYesNoAnswer method



Step into getYesNoAnswer method

The screenshot shows the Eclipse IDE with the following components:

- Editor:** Displays the `UserPrompter.java` file. The `getYesNoAnswer()` method is highlighted, and the `if (answer == null)` condition is being debugged. The `main` method is also visible.
- Debugger:** The `Variables` tab is active, showing the current state of the program:

Name	Value
<code>getAnswer() return</code>	"N" (id=61)
<code>this</code>	UserPrompter (id=4...)
<code>i</code>	2
<code>answer</code>	"N" (id=61)
- Console:** Shows the output of the program:

```
Driver [Java Application] C:\Users\yrfka\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.7.v20250502-0916\jre\bin\javaw.exe (Sep 10, 2025, 8:58:47 PM el...
Seating plan has 14 seats.
Do you want to purchase a ticket? N
Do you want to purchase a ticket?. Please answer Y or N: N
Do you want to purchase a ticket?. Please answer Y or N: . Please answer Y or N: N
```

4) Fix 1 - add condition to handle 'N' explicitly

```
35 public boolean getYesNoAnswer() {
36     for (int i = 0; i < 3; i++) {
37         String answer = getAnswer();
38         if (answer == null)
39             return false;
40         char ans = answer.toUpperCase().charAt(0);
41         if (ans == 'Y')
42             return true;
43         // fix 1 - add condition to handle 'N' explicitly
44         if (ans == 'N')
45             return false;
46         // fix 1 ends
47         setPrompt(getPrompt() + ". Please answer Y or N: ");
48     }
49     return false;
```

4) Step out to main

The screenshot shows the Eclipse IDE with a Java project. The main class is `Manifest.class`. The code is as follows:

```

22 }
23
24 public static void main(String[] args) {
25     System.out.println("Flight open for ticket sales.");
26     SeatingPlan plan = new SeatingPlan();
27     Manifest manifest = new Manifest(plan);
28     SeatReserver reserver = plan.getSeatReserver();
29     int seatsSold = 0;
30     UserPrompter prompter = new UserPrompter("Do you want to purchase a ticket?");
31     while ( prompter.getYesNoAnswer() && seatsSold < plan.getTotalSeats() ) {
32         for ( int c = 0; c < SeatingClass.values().length; c++ ) {
33             SeatingClass sClass = SeatingClass.values()[c];
34             if ( sClass.getNumSeatsSold() < sClass.getNumSeats() ){
35                 UserPrompter prompt2 = new UserPrompter(" Do you want " + sClass + " c
36                     if ( prompt2.getYesNoAnswer() ) {
37                         reserver.sellTicket(sClass);
38                         break;
39                     }
40             }
41         }
42     }
43     manifest.printOut();
44 }

```

A breakpoint is set at line 31. The Variables view on the right shows the following variables:

Name	Value
getYesNoAnswer	false
args	String[0] (id=21)
plan	SeatingPlan (id=23)
manifest	Manifest (id=20)
reserver	SeatReserver (id=26)
seatsSold	0
prompter	UserPrompter (id=4...

The Console view shows the following output:

```

Driver [Java Application] C:\Users\rkfa\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.7.v20250502-0916\jre\bin\javaw.exe (Sep 10, 2025, 8:58:47 PM el
Seating plan has 14 seats.
Do you want to purchase a ticket? N
Do you want to purchase a ticket?. Please answer Y or N: N
Do you want to purchase a ticket?. Please answer Y or N: . Please answer Y or N: N

```

5) Set next breakpoint

The screenshot shows the Eclipse IDE with the same Java project. The code is as follows:

```

23
24 public static void main(String[] args) {
25     System.out.println("Flight open for ticket sales.");
26     SeatingPlan plan = new SeatingPlan();
27     Manifest manifest = new Manifest(plan);
28     SeatReserver reserver = plan.getSeatReserver();
29     int seatsSold = 0;
30     UserPrompter prompter = new UserPrompter("Do you want to purchase a ticket?");
31     while ( prompter.getYesNoAnswer() && seatsSold < plan.getTotalSeats() ) {
32         for ( int c = 0; c < SeatingClass.values().length; c++ ) {
33             SeatingClass sClass = SeatingClass.values()[c];
34             if ( sClass.getNumSeatsSold() < sClass.getNumSeats() ){
35                 UserPrompter prompt2 = new UserPrompter(" Do you want " + sClass + " class? " );
36                 if ( prompt2.getYesNoAnswer() ) {
37                     reserver.sellTicket(sClass);
38                     break;
39                 }
40             }
41         }
42     }
43     manifest.printOut();
44 }
45 }

```

A breakpoint is set at line 37. The Console view shows the following output:

```

Do you want to purchase a ticket? N
Do you want to purchase a ticket?. Please answer Y or N: N
Do you want to purchase a ticket?. Please answer Y or N: . Please answer Y or N: N

```

6) Step into sellTicket method

Code Snippet (SeatReserve.java):

```

89     if (seat == null) {
90         System.out.println("Unable to assign seat, flight closed.");
91         return false;
92     }
93     if (passenger instanceof Discountable) {
94         price = ((Discountable) passenger).discountPrice(price);
95     }
96     Ticket ticket = new Ticket(passenger, seat, price);
97     // important step omitted here
98     System.out.println("Ticket issued: " + ticket);
99     System.out.println();
100    return true;

```

Console Output:

```

UserPrompter [Java Application] C:\Users\rkfa\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.7
Flight open for ticket sales.
Seating plan has 14 seats.
Do you want to purchase a ticket? Y
Do you want BUSINESS class? Y
First name? Garam
Last name? Yoon
Initial? GRY
Are you a frequent flyer? N
Are you an airline employee? Y
Employee ID? 2

```

Variables Window:

Name	Value
discountPrice() retu	375.0
this	SeatReserver (id=27)
sClass	SeatingClass (id=38)
price	375.0
passenger	StaffPassenger (id=10..)
seat	Seat (id=112)

7) Set the breakpoint at line 94, and step into it

Code Snippet (SeatReserve.java):

```

87     Passenger passenger = getPassenger();
88     Seat seat = assignSeat(sClass);
89     if (seat == null) {
90         System.out.println("Unable to assign seat, flight closed.");
91         return false;
92     }
93     if (passenger instanceof Discountable) {
94         price = ((Discountable) passenger).discountPrice(price);
95     }
96     Ticket ticket = new Ticket(passenger, seat, price);
97     // important step omitted here
98     System.out.println("Ticket issued: " + ticket);
99     System.out.println();
100    return true;
101 }
102 }

```

Console Output:

```

UserPrompter [Java Application] C:\Users\rkfa\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.7
Last name? Yoon
Initial? GRY
Are you a frequent flyer? N
Are you an airline employee? Y
Employee ID? 2

```

Variables Window:

Name	Value
no method return value	
this	SeatReserver (id=27)
sClass	SeatingClass (id=35)
price	750.0
passenger	StaffPassenger (id=87)
seat	Seat (id=91)

Code Snippet (StaffPassenger.java):

```

11     super(pName);
12     this.employeeId = employeeId;
13 }
14
15 public double discountPrice(double price) {
16     return price/2.0;
17 }
18
19 public String getEmployeeId() {
20     return employeeId;
21 }
22
23 public void setEmployeeId(String employeeId) {
24     this.employeeId = employeeId;
25 }
26
27

```

Console Output:

```

UserPrompter [Java Application] C:\Users\rkfa\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.7
Last name? Yoon
Initial? GRY
Are you a frequent flyer? N
Are you an airline employee? Y
Employee ID? 2

```

Variables Window:

Name	Value
no method return value	
this	StaffPassenger (id=87)
price	750.0

7) Fix 2 – Add sClass parameter in the discountPrice method, Update the Discountable interface, Add condition inside discountPrice method

```
UserPrompte... Manifest.java UserPrompte... SeatingClas... StaffPasseng... *SeatReserve... × 40
88     Seat seat = assignSeat(sClass);
89     if (seat == null) {
90         System.out.println("Unable to assign seat, flight closed.");
91         return false;
92     }
93     if (passenger instanceof Discountable) {
94         price = ((Discountable) passenger).discountPrice(price, sClass); //fix 2 - deliver Class argument
95     }
96     Ticket ticket = new Ticket(passenger, seat, price);
97     // important step omitted here
98     System.out.println("Ticket issued: " + ticket);
99     System.out.println();
100    return true;
101 }
102 }

1 package com.cc.airline.ticketing;
2
3 public interface Discountable {
4     double discountPrice(double price, SeatingClass sClass); //fix 2
5 }
6
```

Manifest.java SeatingClass... *StaffPasse...

```

8      private String employeeId;
9
10     public StaffPassenger( PassengerName pName, String employeeId) {
11         super(pName);
12         this.employeeId = employeeId;
13     }
14
15     public double discountPrice(double price) {
16         if (sClass == SeatingClass.ECONOMY) {
17             return price / 2.0; // fix 2 - only Economy is discountable
18         }
19         return price;
20     }
21
22     public String getEmployeeId() {
23         return employeeId;
24     }

```

Manifest.java SeatingClass... StaffPasseng... SeatReserve... Discountable... »42

```

89     if (seat == null) {
90         System.out.println("Unable to assign seat, flight closed.");
91         return false;
92     }
93     if (passenger instanceof Discountable) {
94         price = ((Discountable) passenger).discountPrice(price, sClass); //fix 2 - deliver C
95     }
96     Ticket ticket = new Ticket(passenger, seat, price);
97     // important step omitted here
98     System.out.println("Ticket issued: " + ticket);
99     System.out.println();
100    return true;
101 }
102 }

```

Console ×

UserPrompter [Java Application] C:\Users\rkfa\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.7.v20250502-0916

Last name? Yoon
Initial? GRY
Are you a frequent flyer? N
Are you an airline employee? Y
Employee ID? 2
Ticket issued: 1000001 for seat 1A at \$750.0

Name	Value
println() return	(No explicit return ...
> this	SeatReserver (id=...
> sClass	SeatingClass (id=...
> price	750.0
> passenger	StaffPassenger (id...
> seat	Seat (id=91)
> ticket	Ticket (id=105)

<Choose a previously entered expressi>

750.0

8) Set the breakpoint at line 37

StaffPasseng... SeatReserve... Manifest.java × SeatingPlan... »52

```

21     }
22 }
23
24 public static void main(String[] args) {
25     System.out.println("Flight open for ticket sales.");
26     SeatingPlan plan = new SeatingPlan();
27     Manifest manifest = new Manifest(plan);
28     SeatReserver reserver = plan.getSeatReserver();
29     int seatsSold = 0;
30     UserPrompter prompter = new UserPrompter("Do you want to purchase a ti
31     while ( prompter.getYesNoAnswer() && seatsSold < plan.getTotalSeats()
32         for ( int c = 0; c < SeatingClass.values().length; c++ ) {
33             SeatingClass sClass = SeatingClass.values()[c];
34             if ( sClass.getNumSeatsSold() < sClass.getNumSeats() ){
35                 UserPrompter prompt2 = new UserPrompter(" Do you want " +
36                 if ( prompt2.getYesNoAnswer() ) {
37                     reserver.sellTicket(sClass);
38                     break;
39                 }
40             }
41         }
42     }
43     manifest.printOut();
44 }
45 }

```

Console ×

Assignment_Driver [Java Application] C:\Users\rkfa\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86

Flight open for ticket sales.
Seating plan has 14 seats.
Do you want to purchase a ticket? Y
Do you want BUSINESS class? Y

Name	Value
getYesNoAnswer() return	true
> args	String[0] (id=22)
> plan	SeatingPlan (id=24)
> manifest	Manifest (id=26)
> reserver	SeatReserver (id=27)
> seatsSold	0
> prompter	UserPrompter (id=28)
> c	0
> sClass	SeatingClass (id=30)
> prompt2	UserPrompter (id=81)

<Choose a previously entered expression>

9) Step into line 37 and add line, `seat.setTicket(ticket);` before generating (Fix 3)

```
public boolean sellTicket(SeatingClass sClass) {
    double price = sClass.getPrice();
    if (numSeats == numSeatsSold) {
        System.out.println("This flight is sold out.");
        return false;
    }
    Passenger passenger = getPassenger();
    Seat seat = assignSeat(sClass);
    if (seat == null) {
        System.out.println("Unable to assign seat, flight closed.");
        return false;
    }
    if (passenger instanceof Discountable) {
        price = ((Discountable) passenger).discountPrice(price, sClass);
    }
    Ticket ticket = new Ticket(passenger, seat, price);
    seat.setTicket(ticket); //added this line
    System.out.println("Ticket issued: " + ticket);
    System.out.println();
    return true;
}
```

```
Do you want to purchase a ticket? n
Flight manifest
1A Garam GRY Yoon
1B Available
2A Available
2B Available
2C Available
2D Available
3A Available
3B Available
3C Available
3D Available
4A Available
4B Available
4C Available
4D Available
```